Documento de Testes

Relatório dos testes automatizados e matriz de rastreabilidade
Sistema de Batalhas PokeSal

Este documento descreve os testes automatizados do PokeSal: como foram organizados, o que cada classe de teste verifica e o resultado da última execução. No final, a matriz de rastreabilidade liga cada requisito do enunciado e cada requisito autoral aos testes que o verificam.

Ambiente e Execução

Item	Valor
Linguagem	Java 25
Build	Maven
Framework de testes	JUnit 5 (Jupiter) 5.10.2, com testes parametrizados
Mocks	Mockito 5.11.0
Cobertura	JaCoCo 0.8.14
Comando	mvn clean verify
Versão analisada	commit 78f6739

A última execução rodou 192 casos de teste, distribuídos em 101 métodos e 10 classes, sem nenhuma falha, erro ou teste ignorado. A cobertura medida pelo JaCoCo foi de 70,0% das linhas (402 de 574) e 74,3% das condições (150 de 202). O detalhamento da cobertura por pacote está no relatório do SonarQube.

Resultado	Quantidade
Casos executados	192
Falhas	0
Erros	0
Ignorados	0

Estratégia de Testes

Testes unitários. A maior parte dos testes verifica uma classe isolada, chamando seus métodos diretamente e conferindo o resultado. Os PokeSal usados vêm do PokeSalCatalog, o mesmo catálogo usado pelo jogo, para que os valores testados sejam os valores reais.

Mocks. Quando o objetivo é verificar a interação entre objetos, e não o valor final, o Mockito substitui a dependência por um objeto simulado. É o caso do TurnTest, que confere a ordem em que as ações são executadas, e de parte do PassiveTest, que confere quais multiplicadores de buff são aplicados.

Testes parametrizados. Um mesmo método de teste roda com várias entradas, usando @ParameterizedTest com @MethodSource, @CsvSource, @ValueSource ou @NullSource. Isso permite, por exemplo, verificar os seis PokeSal do catálogo, ou as três passivas de HP

baixo, com um único método. Por isso o número de casos (192) é maior que o número de métodos (101).

Valores limite. Os testes de dano, de HP e de ativação das passivas usam valores na fronteira das regras: ATK e DEF iguais a 1, dano igual ao HP e igual ao HP menos 1, e HP exatamente em 1/3 do máximo.

Testes de fluxo da batalha. O RunBattleUseCaseTest executa batalhas completas, com a interface de terminal simulada por mock, para verificar que passivas, condições de status, terrenos e efeitos temporários são acionados nos momentos certos da batalha, e não só quando seus métodos são chamados diretamente.

Testes Exigidos pelo Enunciado

Teste exigido	Implementação
testVantagemElemental()	TypeChartTest.testeVantagemElemental
testEfeitoTerrenoEstacionamentoUCSal()	BattlegroundTest, com os testes de Hot Asphalt, Water Puddle e Central Flowerbed
testOrdemDeAtaquePorVelocidade()	TurnTest.deveExecutarAAcaoDoPokeSal1QuandoMaisRapido e deveExecutarAAcaoDoPokeSal2QuandoMaisRapido
testUsoLimiteDeItensExcedido()	TrainerTest.testUsoLimiteDeItensExcedido
testCalculoDanoBoundaryValues()	DamageCalculatorServiceTest.testCalculoDanoBoundaryValues
Testes dos requisitos autorais (mínimo 2)	PassiveTest (43 casos) e EffectTest (31 casos)

Classes de Teste

Classe	Métodos	Casos	Foco
TypeChartTest	4	11	Tabela de vantagens entre tipos
BattlegroundTest	8	8	Efeitos dos terrenos
TurnTest	7	7	Ordem de ação no turno
PokeSalTest	14	48	HP, buffs, status e validações do PokeSal
TrainerTest	10	12	Mochila e uso de itens
DamageCalculatorServiceTest	9	17	Cálculo de dano
MoveActionTest	6	6	Execução de golpes
PassiveTest	19	43	Habilidades passivas (RA-01 e RA-03)
EffectTest	15	31	Efeitos dos itens (RA-02)
RunBattleUseCaseTest	9	9	Fluxo da batalha
Total	101	192	

TypeChartTest

Teste	Casos	O que verifica
testeVantagemElemental	6	Multiplicador 2.0 nas três vantagens e 0.5 nas três desvantagens entre Fogo, Água e Planta.

Teste	Casos	O que verifica
testeCombinacaoDoMesmoElementoEh Neutra	3	Ataques entre o mesmo tipo têm multiplicador neutro (1.0).
todaCombinacaoDeTiposTem MultiplicadorValido	1	As nove combinações de tipos retornam 0.5, 1.0 ou 2.0.
todaRelacaoCadastradaTemSuaInversa TambemCadastrada	1	Toda relação cadastrada tem a relação inversa também cadastrada.

BattlegroundTest

Teste	Casos	O que verifica
hotAsphaltAumentaSomenteDanosDe GolpesDeFogoEmQuinzePorCento	1	Hot Asphalt aumenta em 15% o dano de golpes de Fogo.
hotAsphaltNaoAlteraDanoDeOutrosTipos	1	Hot Asphalt não altera o dano de golpes de Água e Planta.
waterPuddleAumentaDanoDeAguaEmDezPor Cento	1	Water Puddle aumenta em 10% o dano de golpes de Água.
waterPuddleNaoAlteraDanoDeOutros Tipos	1	Water Puddle não altera o dano de golpes de Fogo e Planta.
centralFlowerbedCuraCincoPorCentoDo HpMaximoDePokeSalDeTipoPlanta	1	Central Flowerbed cura 5% do HP máximo de um PokeSal de Planta no fim do turno.
centralFlowerbedNaoCuraPokeSalDeTipo DiferenteDePlanta	1	Central Flowerbed não cura PokeSal de Fogo nem de Água.
centralFlowerbedNaoCuraAlemDoHp Maximo	1	A cura do Central Flowerbed não ultrapassa o HP máximo.
hotAsphaltEWaterPuddleNaoAlteramHpNo FimDoTurno	1	Hot Asphalt e Water Puddle não alteram o HP no fim do turno.

TurnTest

Teste	Casos	O que verifica
deveExecutarAAcaoDoPokeSal1Quando MaisRapido	1	O primeiro PokeSal age antes quando tem SPD maior, com atacante e alvo corretos.
deveExecutarAAcaoDoPokeSal2Quando MaisRapido	1	O segundo PokeSal age antes quando tem SPD maior.
deveExecutarAAcaoDoPokeSal1Quando VelocidadesForemIguais	1	Em caso de empate de SPD, o PokeSal do primeiro treinador age antes.
naoDeveExecutarAAcaoDoSegundoQuando EleDesmaiar	1	Se o segundo PokeSal desmaia com a primeira ação, a ação dele não é executada.
deveGuardarAOrdemDosAtoresEAcoes	1	O turno registra quem agiu primeiro e depois, e com qual ação.
deveLancarExcecaoQuandoPokeSalFor Nulo	1	Lança NullPointerException quando algum PokeSal é nulo.
deveLancarExcecaoQuandoAcaoForNula	1	Lança NullPointerException quando alguma ação é nula.

PokeSalTest

Teste	Casos	O que verifica
pokeSalNaoDeveEstarComHpNegativo	6	Um dano maior que o HP deixa o PokeSal com 0, nunca negativo, nos seis PokeSal.
pokeSalNaoDeveEstarComHpMaiorQueMaxHp	6	Uma cura maior que o HP perdido não ultrapassa o HP máximo.
healDeveLancarExcecaoSeOValorDeAmountForNegativo	6	Uma cura com valor negativo lança IllegalArgumentException.
takeDamageDeveLancarExcecaoSeOValorDeAmountForNegativo	6	Um dano com valor negativo lança IllegalArgumentException.
buffSeguidoDoInversoDevolveAtributosOriginais	6	Aplicar um buff e depois o multiplicador inverso devolve ATK, DEF e SPD originais.
buffsDevemLancarExcecaoComMultiplicadorNegativo	6	Buffs de ATK, DEF e SPD com multiplicador negativo lançam exceção.
naoDeveAplicarSegundoStatusQuandoJaTemUm	1	Um segundo status é ignorado enquanto o PokeSal já tem um.
clearStatusSemStatusNaoDeveLancarExcecao	1	Limpar o status de um PokeSal sem status não lança exceção.
applyStatusDeveLancarExcecaoComStatusNulo	1	Aplicar um status nulo lança NullPointerException.
receiveEffectDeveLancarExcecaoComEfeitoNulo	1	Receber um efeito nulo lança NullPointerException.
construtorDeveLancarExcecaoComAtributoNaoPositivo	5	HP, ATK, DEF ou SPD iguais a zero ou negativos são rejeitados.
construtorDeveLancarExcecaoComCamposNulos	1	Nome, descrição, tipo ou passiva nulos são rejeitados.
construtorDeveLancarExcecaoSemMovimentos	1	Lista de golpes nula ou vazia é rejeitada.
construtorDeveLancarExcecaoComMaisDeQuatroMovimentos	1	Mais de quatro golpes são rejeitados.

TrainerTest

Teste	Casos	O que verifica
testUsoLimiteDeItensExcedido	1	Com dois itens na mochila, adicionar um terceiro lança IllegalStateException.
mochilaDeveAceitarItensAteOLimite	1	A mochila aceita até dois itens, na ordem em que foram adicionados.
addItemDeveLancarExcecaoComItemNulo	1	Adicionar um item nulo lança exceção.
alterarListaRetornadaNaoAlteraAMochila	1	A lista devolvida por getBackpack é uma cópia da mochila.
construtorDeveLancarExcecaoComNomeVazio	3	Nome nulo, vazio ou só com espaços é rejeitado.
construtorDeveLancarExcecaoComPokeSalNulo	1	PokeSal nulo é rejeitado.
usarItemDeveAplicarEfeitoERemoverDaMochila	1	Usar um item aplica o efeito no PokeSal e remove o item da mochila.

Teste	Casos	O que verifica
naoDeveUsarOMesmoItemDuasVezes	1	O segundo uso do mesmo item lança exceção e não aplica o efeito de novo.
naoDeveUsarItemForaDaMochila	1	Usar um item que não está na mochila lança exceção.
naoDeveUsarItensDepoisDeEsvaziarAMochila	1	Depois de usar os dois itens, nenhum outro uso é aceito.

DamageCalculatorServiceTest

Teste	Casos	O que verifica
testCalculoDanoBoundaryValues	7	O dano segue a fórmula ATK / DEF x poder / 5 com ATK, DEF e poder no mínimo (1), iguais, dobrados e em valores extremos (1000 contra 1).
danoIgualAoHpDeixaDefensorComZero	1	Um dano igual ao HP deixa o defensor com 0.
danoUmAbaixoDoHpDeixaDefensorComUm	1	Um dano igual ao HP menos 1 deixa o defensor com 1.
danoMaiorQueOHpNaoDeixaHpNegativo	1	Um dano maior que o HP não deixa o defensor com HP negativo.
golpeComPoderZeroDeveSerRejeitado	1	Um golpe com poder 0 é rejeitado.
danoDeveAplicarMultiplicadorDeTipo	3	O dano é multiplicado por 2.0, 0.5 ou 1.0 conforme a relação entre os tipos.
tipoTerrenoESolarPowerSeAcumulam	1	Tipo, Hot Asphalt e Solar Power se multiplicam: 10 x 2.0 x 1.15 x 1.5 = 34.5.
hotAsphaltSemSolarPowerAplicaSoOBonusDoTerreno	1	No Hot Asphalt, uma passiva sem bônus de ataque recebe só o bônus do terreno.
deveLancarExcecaoComArgumentosNulos	1	Atacante, defensor, golpe ou terreno nulos lançam exceção.

MoveActionTest

Teste	Casos	O que verifica
golpeComChanceTotalAplicaOStatus	1	Um golpe com chance de status 1.0 aplica o status no defensor.
golpeComChanceZeroNuncaAplicaOStatus	1	Um golpe com chance de status 0.0 não aplica o status.
golpeSemStatusNaoAplicaNenhumStatus	1	Um golpe sem status não aplica nenhum.
golpeQueDerrubaNaoAplicaStatus	1	Um golpe que derruba o defensor não aplica status nele.
golpeAvisaAPassivaDoDefensorSobreODano	1	O dano recebido aciona a passiva do defensor (Blaze ativa abaixo de 1/3 do HP).
construtorDeveLancarExcecaoComGolpeNulo	1	Golpe nulo é rejeitado.

PassiveTest

Teste	Casos	O que verifica
blazeDeveAtivarQuandoHpAbaixoDeUm Terco	1	Blaze aumenta o ATK do CharSal em 50% quando o HP cai abaixo de 1/3.
buffDeveAtivarQuandoHpCruzaAbaixoDe UmTerco	3	Blaze, Overgrow e Torrent aplicam o buff de 1.5 quando o HP cruza abaixo de 1/3.
buffNaoDeveAtivarComHpExatamenteEmUm Terco	3	O buff não ativa com HP exatamente em 1/3.
buffNaoDeveAcumularComDanosSeguidos	3	Danos seguidos abaixo de 1/3 não aplicam o buff mais de uma vez.
buffDeveDesativarQuandoHpVoltaASubir	3	O buff é revertido quando o HP volta a ficar acima de 1/3.
buffDeveReativarDepoisDeDesativado	3	O buff pode ativar de novo depois de desativado.
buffNaoDeveAtivarEmPokeSalDeOutro Tipo	3	Cada passiva só ativa em PokeSal do seu tipo elemental.
intimidateReduzAtkDoOponenteAoEntrar EmCampo	1	Intimidate reduz o ATK do oponente ao entrar em campo.
intimidateNaoMexeNoProprioPokeSal	1	Intimidate não altera o próprio PokeSal.
rainDishAumentaDefEmVintePorCentoAo EntrarEmCampo	1	Rain Dish aumenta a DEF do próprio PokeSal em 20% ao entrar em campo.
leafGuardImpedeAplicacaoDeStatus	1	Leaf Guard impede a aplicação de Poison, Burn e Paralyzed.
passivasDiferentesDeLeafGuardNao BloqueiamStatus	6	As outras seis passivas não bloqueiam status.
solarPowerAumentaAtaqueEmHotAsphalt	1	Solar Power retorna multiplicador 1.5 ao atacar no Hot Asphalt.
solarPowerNaoAlteraAtaqueForaDeHot Asphalt	2	Solar Power retorna 1.0 no Water Puddle e no Central Flowerbed.
solarPowerCobraHpNoFimDoTurnoQuando Ativo	1	Com o bônus ativo, Solar Power cobra 12,5% do HP máximo no fim do turno.
solarPowerNaoCobraHpForaDeHotAsphalt	2	Solar Power não cobra HP depois de atacar fora do Hot Asphalt.
solarPowerNaoCobraHpSemTerAtacado	1	Solar Power não cobra HP se o PokeSal não atacou.
solarPowerDesativaQuandoOTerrenoMuda	1	O bônus deixa de valer quando o ataque acontece em outro terreno.
passivasSemBonusNaoAlteramDanoEm NenhumTerreno	6	As outras passivas retornam multiplicador 1.0 em todos os terrenos.

EffectTest

Teste	Casos	O que verifica
deveCurarPoucoSemStatus	3	Super Antidote, Burn Heal e Paralyze Heal curam 10% do HP máximo quando o PokeSal não tem status.
deveCurarMaisERemoverStatus Correspondente	3	Os três itens curam 25% e removem o status correspondente.

Teste	Casos	O que verifica
deveCurarPoucoQuandoStatusFor Diferente	6	Com um status diferente do que o item cura, a cura é de 10% e o status continua.
burnHealDevolveOAtkReduzidoPela Queimadura	1	Ao remover a queimadura, o ATK volta ao valor original.
paralyzeHealDevolveOSpdReduzidoPela Paralisia	1	Ao remover a paralisia, o SPD volta ao valor original.
curaForteNaoPassaDoHpMaximo	3	A cura de 25% não ultrapassa o HP máximo.
healEffectCuraAPorcentagemDoHpMaximo	1	HealEffect cura a porcentagem configurada do HP máximo.
healEffectNaoPassaDoHpMaximo	1	A cura do HealEffect não ultrapassa o HP máximo.
healEffectNaoRemoveStatus	1	HealEffect não remove o status do PokeSal.
healEffectDeveLancarExcecaoComCura NaoPositiva	3	Porcentagem de cura igual a zero ou negativa é rejeitada.
speedBuffEffectAumentaSpdEmTrintaPor Cento	1	SpeedBuffEffect aumenta o SPD em 30%.
speedBuffEffectRemoveAplicando OMultiplicadorInverso	1	A reversão do SpeedBuffEffect aplica o multiplicador inverso.
speedBuffEffectEhTemporario	1	SpeedBuffEffect é um efeito temporário.
speedBuffEffectExpiraNoTurnoSeguinte	1	O efeito fica agendado para o turno seguinte e é revertido ao expirar.
efeitosDeCuraNaoSaoTemporarios	4	Os efeitos de cura são instantâneos e não ficam agendados.

RunBattleUseCaseTest

Teste	Casos	O que verifica
intimidateAgeAoEntrarEmCampo	1	Intimidate reduz o ATK do oponente antes do primeiro turno.
rainDishAgeAoEntrarEmCampo	1	Rain Dish aumenta a DEF antes do primeiro turno.
venenoCausaDanoNoFimDoTurno	1	Poison causa dano no fim de cada turno.
venenoPodeDecidirABatalha	1	A batalha termina quando o veneno derruba um PokeSal no fim do turno.
centralFlowerbedCuraNoFimDoTurno	1	Central Flowerbed cura o PokeSal de Planta no fim do turno.
centralFlowerbedNaoCuraPokeSal Derrubado	1	Central Flowerbed não cura um PokeSal derrubado pelo veneno.
blazeAtivaComDanoDoVeneno	1	Blaze ativa quando o dano do veneno deixa o HP abaixo de 1/3.
speedBuffExpiraNoFimDoTurnoSeguinte	1	O buff de velocidade dura até o fim do turno seguinte ao uso.
deveTerminarNoPrimeiroTurnoQuando AlguemEhNocauteado	1	A batalha termina no turno em que um PokeSal é nocauteado.

Matriz de Rastreabilidade

A matriz abaixo liga cada requisito aos testes que o verificam. Os requisitos REQ-01 a REQ-05 correspondem aos itens 1 a 5 do enunciado, e RA-01 a RA-03 aos requisitos autorais do grupo. A coluna Situação indica se o requisito está totalmente coberto por testes ou se alguma parte ficou sem teste automatizado.

ID	Requisito	Testes	Situação
REQ-01	Seleção do PokeSal inicial entre seis opções, cada uma com HP, ATK, DEF, SPD e tipo elemental.	PokeSalTest: testes parametrizados sobre os seis PokeSal do catálogo e validações do construtor.	Parcial: a escolha do PokeSal pelo terminal não tem teste automatizado.
REQ-02	Matriz de vantagens e desvantagens entre Fogo, Água e Planta.	TypeChartTest: todos os testes. DamageCalculatorServiceTest.danoDeveAplicarMultiplicadorDeTipo.	Coberto
REQ-03	Efeito de terreno do estacionamento: Hot Asphalt, Water Puddle e Central Flowerbed.	BattlegroundTest: todos os testes. RunBattleUseCaseTest.centralFlowerbedCuraNoFimDoTurno e centralFlowerbedNaoCuraPokeSalDerrubado. DamageCalculatorServiceTest.hotAsphaltSemSolarPowerAplicaSoOBonusDoTerreno.	Coberto
REQ-04a	Iniciativa por SPD: o PokeSal mais rápido age primeiro.	TurnTest: deveExecutarAAcaoDoPokeSal1QuandoMaisRapido, deveExecutarAAcaoDoPokeSal2QuandoMaisRapido, deveExecutarAAcaoDoPokeSal1QuandoVelocidadesForemIguais, naoDeveExecutarAAcaoDoSegundoQuandoEleDesmaiar.	Coberto
REQ-04b	Cálculo de dano e fim da batalha quando um PokeSal chega a 0 de HP.	DamageCalculatorServiceTest: testCalculoDanoBoundaryValues, danoIgualAoHpDeixaDefensorComZero, danoUmAbaixoDoHpDeixaDefensorComUm, danoMaiorQueOHpNaoDeixaHpNegativo. PokeSalTest.pokeSalNaoDeveEstarComHpNegativo. RunBattleUseCaseTest.deveTerminarNoPrimeiroTurnoQuandoAlguemEhNocauteado.	Coberto
REQ-04c	Condições de status: Burn, Poison e Paralyzed.	RunBattleUseCaseTest: venenoCausaDanoNoFimDoTurno, venenoPodeDecidirABatalha. MoveActionTest: golpeComChanceTotalAplicaOStatus, golpeComChanceZeroNuncaAplicaOStatus. PokeSalTest.naoDeveAplicarSegundoStatusQuandoJaTemUm. EffectTest: burnHealDevolveOAtkReduzidoPelaQueimadura, paralyzeHealDevolveOSpdReduzidoPelaParalisia.	Parcial: o dano por turno da Burn e as reduções de ATK e SPD ao aplicar Burn e Paralyzed não têm teste direto; os testes verificam só a reversão.

ID	Requisito	Testes	Situação
REQ-05	Mochila com no máximo dois itens por treinador por batalha.	TrainerTest: testUsoLimiteDeItensExcedido, mochilaDeveAceitarItensAteOLimite, usarItemDeveAplicarEfeitoERemoverDa Mochila, naoDeveUsarOMesmoItemDuasVezes, naoDeveUsarItemForaDaMochila, naoDeveUsarItensDepoisDeEsvaziar AMochila. EffectTest: testes do HealEffect.	Coberto
RA-01	Habilidades passivas: Blaze, Overgrow, Torrent, Intimidate, Rain Dish, Leaf Guard e Solar Power.	PassiveTest: todos os testes. RunBattleUseCaseTest: intimidateAgeAoEntrarEmCampo, rainDishAgeAoEntrarEmCampo, blazeAtivaComDanoDoVeneno. MoveActionTest.golpeAvisaAPassivaDo DefensorSobreODano.	Coberto
RA-02	Itens que reagem a um status presente: Super Antidote, Burn Heal e Paralyze Heal.	EffectTest: deveCurarPoucoSemStatus, deveCurarMaisERemoverStatus Correspondente, deveCurarPoucoQuandoStatusFor Diferente, curaForteNaoPassaDoHpMaximo, burnHealDevolveOAtkReduzidoPela Queimadura, paralyzeHealDevolveOSpdReduzidoPela Paralisia.	Coberto
RA-03	Sinergia entre terreno e habilidade: Solar Power no Hot Asphalt.	PassiveTest: testes da Solar Power. DamageCalculatorServiceTest.tipo TerrenoESolarPowerSeAcumulam.	Coberto

Fase 03 — Testes com Mockito

Atualização do documento de testes para a Fase 03

As páginas anteriores descrevem os testes da Fase 02 e continuam valendo. Esta parte descreve o que mudou na Fase 03: os testes que isolam cada classe dos seus colaboradores com o Mockito, os testes exigidos pelo enunciado, a matriz de rastreabilidade das regras novas e dos requisitos autorais RA-04 e RA-05, e a justificativa de quais colaboradores foram mockados e quais não foram.

Ambiente e Execução

Item	Valor
Linguagem	Java 25
Build	Maven
Framework de testes	JUnit 5 (Jupiter) 5.10.2
Mocks	Mockito 5.11.0, com @ExtendWith(MockitoExtension.class)
Cobertura	JaCoCo 0.8.14, com meta verificada no build (80% de linhas e 70% de branches nas classes de regra de negócio)
Análise estática	Checkstyle (estilo Google) e SonarQube Community 26.11
Integração contínua	GitHub Actions, a cada push na main e a cada pull request
Comando	mvn clean verify
Versão analisada	commit 15e2212

A última execução rodou 290 casos de teste, distribuídos em 178 métodos e 25 classes, sem nenhuma falha, erro ou teste ignorado. Na Fase 02 eram 192 casos em 10 classes. A cobertura das classes de regra de negócio foi de 91,1% das linhas (655 de 719) e 90,6% dos branches (269 de 297). O relatório completo do JaCoCo está em docs/relatorio-jacoco.

Testes Exigidos pelo Enunciado da Fase 03

Os oito testes obrigatórios foram implementados com os nomes exatos do enunciado, na classe de teste da regra que cada um verifica.

Teste exigido	Classe	Colaboradores mockados	O que valida
testAcertoCriticoComGeradorMockado	DamageCalculatorServiceTest	RandomGenerator, BattlegroundService	Com o gerador devolvendo 0.99 e depois 0.05, o primeiro ataque causa 10 de dano e o segundo, crítico, causa 15.
testGolpeErraQuandoPrecisaoFalha	MoveActionTest	RandomGenerator, BattlegroundService, BattleLogger	Golpe com precisão 70 e sorteio 70: dano zero, HP do alvo inalterado, logMiss chamado e logDamage nunca chamado.
testAsfaltoQuenteApenasDuranteODia	DefaultBattlegroundServiceTest	Clock, WeatherProvider	Com o relógio às 12h e depois às 22h, o Hot Asphalt fica ativo e depois inativo.

Teste exigido	Classe	Colaboradores mockados	O que valida
testSequenciaDeEventosNoTurno	TurnTest	BattleLogger, RandomGenerator, BattlegroundService	Num turno real com ataque e item, o registro recebe ataque, dano, status, terreno e item nessa ordem (InOrder).
testResultadoSalvoAoFimDaBatalha	RunBattleUseCaseTest	ResultRepository, BattleCli, BattleLogger	ArgumentCaptor do BattleResult salvo: vencedor, perdedor, 2 turnos e os PokeSal.
testFalhaNoRepositorioNaoInvalidaBatalha	RunBattleUseCaseTest	ResultRepository, BattleLogger, BattleCli	O repositório lança exceção (doThrow): a batalha continua com vencedor e o erro é registrado.
testRepositorioNaoChamadoSeBatalhaNaoTerminou	RunBattleUseCaseTest	ResultRepository, BattleCli	A CLI lança exceção no segundo turno (thenThrow): o repositório nunca é chamado (never).
testChaveamentoTorneioAvancaVencedores	RunTournamentUseCaseTest	MatchRunner, BattleCli	Resultados mockados de cada batalha definem quem avança; o campeão e a ordem dos confrontos são conferidos.

Recursos do Mockito Aplicados

Recurso	Onde é aplicado
@Mock e @InjectMocks	@Mock em todas as classes que usam colaboradores mockados; @InjectMocks em DamageCalculatorServiceTest, DefaultBattlegroundServiceTest, ActionTimeLimitTest, CriticalHpAlertTest, DefaultMatchRunnerTest, RunTournamentUseCaseTest, TurnActionFactoryTest e nos testes dos casos de uso de cadastro.
when(...).thenReturn(...) com retornos em sequência	testAcertoCriticoComGeradorMockado (0.99 e 0.05), testAsfaltoQuenteApenasDuranteODia (12h e 22h), ActionTimeLimitTest (instantes antes e depois da escolha) e ChooseItemsUseCaseTest (itens escolhidos em sequência).
verify com times(), never() e atLeastOnce()	times(7) nas batalhas do torneio de 8 treinadores; never() no repositório e no notificador; atLeastOnce() na renderização do estado da batalha em testResultadoSalvoAoFimDaBatalha.
InOrder	testSequenciaDeEventosNoTurno, testChaveamentoTorneioAvancaVencedores e venenoNoFimDoTurnoDisparaAlerta (dano do veneno registrado antes do alerta).
ArgumentCaptor	testResultadoSalvoAoFimDaBatalha (BattleResult), alertaEnviadoQuandoHpFicaCritico (treinador e PokeSal alertados) e batalhaUsaTreinadoresETerrenoSorteado (Battle montada pelo DefaultMatchRunner).
thenThrow(...) e doThrow(...)	thenThrow na CLI em testRepositorioNaoChamadoSeBatalhaNaoTerminou; doThrow no repositório em testFalhaNoRepositorioNaoInvalidaBatalha e falhaNoRepositorioNaoInterrompeTorneio, e no notificador em falhaNoNotificadorNaoInterrompeBatalha.
Argument matchers (any(), eq())	logDamage(eq(squirtSal), anyDouble(), eq(false)) em testSequenciaDeEventosNoTurno; announceMatch(eq("Final"), any(), any()) no torneio; execute(any(Battle.class)) no DefaultMatchRunnerTest.

Matchers ou Valores Exatos

A regra seguida nos testes foi usar valores exatos sempre que o teste conhece o objeto esperado e esse valor faz parte da regra verificada. Por exemplo, verify(trainerNotifier).notifyCriticalHp(trainerB, squirtSal) usa valores exatos porque o alerta precisa ir para o treinador B e para o PokeSal dele, e não para qualquer um. Do mesmo jeito, nextInt(100) é configurado com o valor exato porque o sorteio de precisão precisa usar o intervalo de 0 a 99.

Os matchers foram usados em três situações. A primeira é quando o objeto é criado dentro da classe testada e o teste não tem referência a ele, como a Battle que o DefaultMatchRunner monta, configurada com any(Battle.class). A segunda é quando o argumento não importa para o que está sendo verificado: em never().save(any()), o que importa é que nada foi salvo, qualquer que fosse o resultado. A terceira é quando parte dos argumentos importa e parte não: em logDamage(eq(squirtSal), anyDouble(), eq(false)), o teste de ordem confere o alvo e que o golpe não foi crítico, mas não o valor do dano, que já é coberto pelos testes de cálculo. Nesses casos, como o Mockito não permite misturar matchers e valores, os valores exatos ficam dentro de eq().

Quando o objeto criado internamente precisa ser inspecionado, e não apenas aceito, o teste usa ArgumentCaptor no lugar do matcher, como no BattleResult salvo no repositório.

Matriz de Rastreabilidade da Fase 03

A matriz liga cada regra nova do enunciado da Fase 03 (itens 2.1 a 2.5) e cada requisito autoral da fase aos casos de teste que a verificam, aos colaboradores mockados nesses testes e ao resultado esperado e obtido. Os requisitos REQ-01 a REQ-05 e RA-01 a RA-03, da Fase 02, estão na matriz das páginas anteriores e continuam cobertos pelos mesmos testes, que foram adaptados aos novos construtores.

ID	Requisito	Casos de teste	Colaboradores mockados	Resultado esperado	Resultado obtido
REQ-06	Precisão do golpe: um golpe pode errar e, nesse caso, o dano é zero (2.1).	MoveActionTest.testGolpeErraQuandoPrecisaoFalha; MoveTest (7 métodos, 11 casos) para a validação e os limites da precisão.	RandomGenerator, BattlegroundService, BattleLogger	Dano zero, HP do alvo inalterado e erro registrado; precisão fora de 0 a 100 rejeitada.	Passou (12 casos)
REQ-07	Acerto crítico multiplica o dano por 1.5, com chance de 6,25% (2.1).	DamageCalculatorServiceTest.testAcertoCriticoComGeradorMockado e criticoRespeitaOLimiteDaFaixa.	RandomGenerator, BattlegroundService	Dano de 10 vira 15 no crítico; 0.0624 é crítico e 0.0625 não é.	Passou (5 casos)
REQ-08	Toda aleatoriedade vem de uma interface (2.1).	RandomWeatherProviderTest (3 métodos), DefaultRandomGeneratorTest (2 métodos) e DefaultMatchRunnerTest.batalhaUsaTreinadoresETerrenoSorteado.	RandomGenerator	O clima e o terreno seguem o índice sorteado pelo gerador; o clima é sorteado uma única vez.	Passou (8 casos)
REQ-09	Efeito do terreno depende de horário e clima, com o horário vindo de um Clock (2.2).	DefaultBattlegroundServiceTest.testAsfaltoQuenteApenasDuranteODia, asfaltoQuenteRespeitaOsLimitesDoDia, pocaDeAguaDependeDoClima e canteiroCentralSempreAtivo.	Clock, WeatherProvider	Hot Asphalt ativo das 6h às 17h59; Water Puddle inativa com sol; Central Flowerbed sempre ativo.	Passou (9 casos)
REQ-10	Torneio eliminatório com 4 ou 8 treinadores (2.3).	RunTournamentUseCaseTest.testChaveamentoTorneioAvancaVencedores, torneioComOitoTreinadoresTemTresFases, torneioRejeitaQuantidadeInvalida, torneioRejeitaListaNula e vencedorForaDoConfrontoEhRejeitado; DefaultMatchRunnerTest.	MatchRunner, BattleCli, RunBattleUseCase, RandomGenerator	Vencedores avançam até o campeão; 7 batalhas com 8 treinadores; outras quantidades rejeitadas; PokeSal restaurados antes de cada batalha.	Passou (14 casos)
REQ-11	Resultado salvo no repositório ao fim de cada batalha, com vencedor, perdedor, turnos e PokeSal (2.3).	RunBattleUseCaseTest.testResultadoSalvoAoFimDaBatalha e testRepositorioNaoChamadoSeBatalhaNaoTerminou; InMemoryResultRepositoryTest.	ResultRepository, BattleCli, BattleLogger	Um único save com os dados certos; nenhum save se a batalha não terminou.	Passou (6 casos)

ID	Requisito	Casos de teste	Colaboradores mockados	Resultado esperado	Resultado obtido
REQ-12	Falha do repositório não invalida a batalha: o erro é registrado e o torneio continua (2.3).	RunBattleUseCaseTest.testFalhaNoRepositorioNaoInvalidaBatalha; RunTournamentUseCaseTest.falhaNoRepositorioNaoInterrompeTorneio.	ResultRepository, BattleLogger, BattleCli, Clock	Vencedor definido, logError chamado e torneio concluído com campeão.	Passou (2 casos)
REQ-13	Eventos do turno enviados ao registro na ordem ataque, dano, status, terreno e item (2.4).	TurnTest.testSequenciaDeEventosNoTurno; RunBattleUseCaseTest.inicioDaBatalhaEhRegistrado, danoDoVenenoEhRegistrado e statusSemDanoNaoEhRegistradoNoFimDoTurno.	BattleLogger, RandomGenerator, BattlegroundService	Ordem conferida com InOrder e nenhum evento além dos esperados.	Passou (4 casos)
REQ-14	Colaboradores recebidos pelo construtor, sem serem instanciados nas classes de regra de negócio (2.5).	Testes de construtor com colaboradores nulos (construtorRejeitaNulos e equivalentes) e todas as classes que usam @InjectMocks.	Todos os colaboradores da tabela de justificativa	Construtores rejeitam nulos; as classes testadas funcionam com mocks no lugar das implementações reais.	Passou
RA-04	Tempo limite de 30 segundos para escolher a ação; quem passa do limite perde a vez.	ActionTimeLimitTest: treinadorQuePassaDoLimitePerdeAVez, escolhaDentroDoLimiteEhExecutada, itemEscolhidoForaDoLimiteNaoEhConsumido e tempoLimiteEhMedidoPorTreinador.	Clock (retornos em sequência), BattleCli, BattleLogger	Com 31 s a ação é descartada e o tempo esgotado é registrado; 0, 10 e 30 s valem; o item não é consumido; o limite vale por treinador.	Passou (6 casos)
RA-05	Alerta de HP crítico ao treinador quando o HP fica abaixo de 25%, uma vez por batalha.	CriticalHpAlertTest (7 métodos); ConsoleTrainerNotifierTest.	TrainerNotifier, BattleLogger, BattleCli, Clock	Um alerta ao cruzar o limite; sem repetição, nem depois de cura; sem alerta em nocaute direto nem com exatamente 25%; falha registrada sem parar a batalha.	Passou (9 casos)

Justificativa dos Mocks

Um colaborador foi mockado quando ele traz para o teste algo que a classe testada não controla: sorteio, horário, armazenamento, saída no console ou uma batalha inteira. Com o mock, o teste escolhe o valor que o colaborador devolve e confere como a classe testada conversa com ele.

Colaborador mockado	Por que foi mockado
RandomGenerator	Os sorteios de precisão, crítico, status, clima e terreno mudam a cada execução. O mock fixa o valor sorteado e permite testar os limites exatos, como 0.0624 e 0.0625.
Clock	O resultado dependeria da hora em que o teste roda. O mock define o horário (dia ou noite) e, com retornos em sequência, simula quanto tempo o jogador levou para escolher a ação, sem esperar 30 segundos de verdade.

Colaborador mockado	Por que foi mockado
WeatherProvider	O clima real é sorteado. O mock define o clima de cada teste.
BattlegroundService	Nos testes de dano e de golpe, o interesse não é a regra do terreno, que tem testes próprios. O mock decide se o terreno está ativo, sem depender de relógio e clima.
ResultRepository	Permite conferir o resultado salvo (ArgumentCaptor), garantir que nada foi salvo (never) e simular uma falha de persistência (doThrow).
BattleLogger	O registro é uma saída da classe testada, e não um valor de retorno. O mock permite verificar quais eventos foram enviados e em que ordem (InOrder).
TrainerNotifier	Mesmo motivo do registro: o alerta é uma interação com outro objeto. O mock conta os alertas, captura os argumentos e simula falha no envio.
BattleCli	Lê o teclado com Scanner. O mock devolve as ações e escolhas do jogador sem entrada de terminal, inclusive em sequência e com exceção (thenThrow).
MatchRunner	Uma batalha real depende de turnos, sorteios e entradas do jogador. O mock define o resultado de cada confronto, e o teste do torneio confere só o chaveamento.
RunBattleUseCase e DamageCalculatorService	No DefaultMatchRunnerTest e no TurnActionFactoryTest, a classe testada só repassa o trabalho para eles. O mock confere o que foi repassado sem executar uma batalha ou um cálculo completo.
PokeSal (TurnTest, PassiveTest e EffectTest)	Nesses testes, o que importa é a velocidade ou o HP em um valor específico, e a interação com os métodos de buff. O mock fixa esses valores e permite verificar os multiplicadores aplicados.
TurnAction e Battleground (TurnTest)	O teste de ordem por velocidade confere só quem age primeiro. As ações e o terreno mockados permitem verificar a ordem das chamadas sem executar golpes nem efeitos de terreno.

Algumas classes foram mantidas reais de propósito. A própria classe sob teste nunca é mockada, já que mocká-la faria o teste verificar o mock e não o código. O PokeSal é real na maior parte dos testes, criado pelo PokeSalCatalog: é uma entidade sem dependências externas, e usar os PokeSal do catálogo garante que os valores testados são os do jogo. A TypeChart, a matriz de tipos, também é real, porque é uma tabela fixa e determinística, e mocká-la deixaria os testes de dano verificando valores inventados. O mesmo vale para Move, Item, Trainer, Battle, os terrenos e as condições de status, que são objetos de regra sem efeitos externos. No teste de ordem dos eventos, o DamageCalculatorService, o MoveAction e o ItemAction também são reais, para que a ordem verificada seja a do código de produção; só os colaboradores externos deles foram mockados.

Registro de Bugs

Os bugs encontrados na Fase 03 estão no documento de checklist estático e registro de bugs, na página do registro da Fase 03, com o commit de cada correção: BUG-07 (registro de status repetido e ambíguo, corrigido no commit 4abdf94) e BUG-08 (violações do Checkstyle que passaram despercebidas, corrigidas no commit 6cab06c). Os testes com Mockito escritos para as etapas 4.3 e 4.4 não revelaram nenhum bug novo nas regras de negócio.